

Contents

guide Let's Verify Step by Step	1
0. 这篇论文到底在改写什么	1
1. 论文结构地图	2
2. 历史背景: 为什么 outcome supervision 不够	2
3. 三个核心对象: generator, ORM, PRM	3
4. 数据: PRM800K 到底是什么	3
5. Active learning 的真实含义	4
6. PRM 的训练目标	4
7. 从 step score 到 solution score	5
8. 为什么只监督到第一处错误	6
9. Best-of-N: reward model 质量如何被放大	6
10. Large-scale 结果: 主证据是什么	7
11. Small-scale synthetic: 更公平的消融	7
12. OOD 和限制: 论文证明了什么, 没有证明什么	8
13. Alignment 论证: negative alignment tax	8
14. 方法总图	8
15. 张量级别图示	9
16. 代码样例: 最小 PRM 聚合	10
17. 和 Math-Shepherd、PRIME、RLVR 的关系	10
18. 和本仓库的连接	11
19. AI agent 应该怎样辅助你学这篇	11
20. 常见误读	12
21. 现代意义	12
22. 闭卷掌握检查	12

guide_Let's Verify Step by Step

原论文: Let's Verify Step by Step
本地原文 PDF: [learning/process-reward/paper/01_lets_verify_step_by_step.pdf](#)
作者: Lightman et al., OpenAI
年份: 2023
类型: paper

0. 这篇论文到底在改写什么

这篇论文的核心主张是: 对复杂数学推理, 只看最终答案是否正确的 outcome supervision 不够细; 如果让人类逐步检查推理过程, 并训练一个 Process Reward Model, 简称 PRM, 去判断每一步是否合理, 那么这个 reward model 会比只看最终答案的 Outcome Reward Model, 简称 ORM, 更可靠。

注意它不是在训练一个更强的 generator。论文明确把范围限定在 reward model: 固定一个 generator, 从它那里采样很多条解题路径, 然后让 reward model 做 best-of-N reranking。最终指标是: 在 N 条候选解里, reward model 能不能挑中最终答案正确的那一条。

它的重要性在于把一个后来的大方向说清楚了: LLM reasoning 不仅需要会生成 chain-of-thought, 还需要一个能检查 chain-of-thought 的机制。Outcome reward 问的是 “答案对不对”, process reward 问的是 “这一步为什么对, 错在哪里”。后者更贵, 但信息密度更高。

读这篇论文时要抓住四个关键词:

1. Credit assignment: 最终错了, 不代表每一步都错; ORM 必须猜错误发生在哪里。
2. Step-level supervision: PRM 在每个推理步骤结束处预测 positive、negative、neutral。
3. Best-of-N search: reward model 不是直接生成答案, 而是在许多候选答案里排序。
4. Active learning: 最值得标注的不是随机样本, 而是“看起来很像对、但最终答案错”的样本。

1. 论文结构地图

原论文可以按下面顺序读:

1. Abstract 和 Introduction: 给出主结论, PRM 在 MATH 上明显优于 ORM, 并发布 PRM800K。
2. Methods: 解释 large-scale 和 small-scale 两套实验设置, 说明 generator 固定, 不做 RL。
3. Data Collection: 解释人类如何给每一步标 positive、negative、neutral, 以及为什么要主动采集 convincing wrong-answer solutions。
4. Outcome Reward Models: 说明 ORM 如何从最终答案正确与否学习一个 solution-level 分数。
5. Process Reward Models: 说明 PRM 如何在每一步结束位置做分类, 并把 step scores 聚合成 solution score。
6. Large-scale Supervision: 主结果, PRM best-of-1860 达到 78.2%, ORM 为 72.4%, majority voting 为 69.6%。
7. Small-scale Synthetic Supervision: 用 large PRM 替代人类, 做更公平的过程监督和结果监督对比。
8. Active Learning: 说明主动选择高分但答案错的样本, 约带来 2.6x 数据效率提升。
9. OOD Generalization: 在近期 STEM 题上验证 PRM 相对 ORM 和 majority voting 仍有优势。
10. Discussion: 讨论 credit assignment、alignment impact、test set contamination 和泛化边界。
11. Appendices: 很关键, 里面有 PRM800K 采集细节、neutral 标签处理、PRM 打分策略、synthetic supervision 的 threshold。

如果时间有限, 优先精读 Methods、Data Collection、Process Reward Models、Figure 3、Figure 4、Appendix F、Appendix H。Appendix F 告诉你 PRM 到底怎么训练和打分, Appendix H 告诉你小规模实验里的“合成标签”是怎么来的。

2. 历史背景: 为什么 outcome supervision 不够

在这篇论文之前, 数学推理方向已经有几条重要线索:

- Chain-of-thought prompting 让模型把中间推理写出来。
- Self-consistency 和 majority voting 让模型多采样几次, 再用投票提高准确率。
- Cobbe et al. 的 verifier 思路训练一个模型挑选多个答案中的正确答案。
- RLHF 和 preference modeling 证明了 reward model 可以指导生成模型行为。

这些方法共同暴露了一个问题: 如果一个题有五步推理, 最后答案错了, outcome label 只告诉你“整条解法错”。它没有告诉你:

- 前两步是不是其实正确。
- 第三步是不是第一次出错。
- 第四步是不是在错误前提上继续推导。
- 最后答案如果碰巧对了, 中间推理是否仍然错误。

这就是 credit assignment 难题。Outcome supervision 把一整条解法压成一个 bit, 而 process supervision 把一条解法拆成一串局部判断。对学习 reward model 来说, 这相当于从“全卷只给一个总分”变成“每一步批改并指出第一处错误”。

论文的直觉很朴素: 数学推理的错误往往发生在某个明确步骤。只要人类能指出那个步骤, reward model 就不必自己从最终错答反推错误位置。这个额外信号很贵, 但对难题尤其值钱。

3. 三个核心对象: generator, ORM, PRM

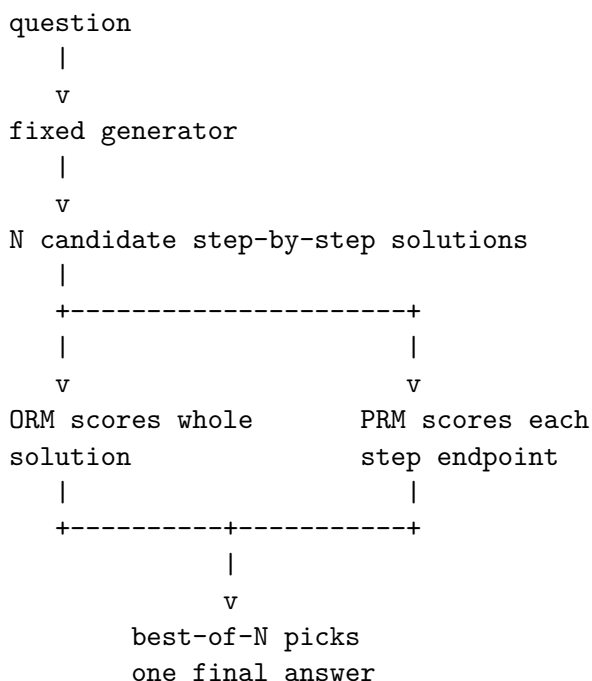
论文里有三个角色, 千万不要混在一起。

Generator 是解题模型。它负责为每道 MATH 题采样多条 step-by-step solutions。论文没有用 reward model 对 generator 做 RL 训练。Generator 只是产生候选解。

ORM 是 outcome-supervised reward model。它看到一整条 solution, 然后学习这条 solution 的最终答案是否正确。测试时通常用最后一个 token 的 reward score 当作整条解法分数。

PRM 是 process-supervised reward model。它看到问题和部分解法, 在每个 step 的结束 token 上预测这一步是 positive、negative 还是 neutral。测试时只需要对整条 solution 做一次 forward pass, 就可以拿到所有 step-end 的分类概率。

它们的关系可以画成:



这个图要读成一句话: 论文比较的不是“谁生成得更好”, 而是“在同一批候选解里, 谁挑得更准”。

4. 数据: PRM800K 到底是什么

PRM800K 是这篇论文最重要的工程资产之一。论文报告的原始采集规模约为 1,085,590 个 step-level labels, 覆盖 101,599 条 solution samples。训练时过滤掉质量控制标签和未完成任务的标签后, 得到约 800,000 个 step-level labels, 覆盖约 75,000 条 solutions 和 12,000 个 problems。

每一步有三个可能标签:

- positive: 这一步正确、合理, 并且推进了解题。
- negative: 这一步错误或不合理。
- neutral: 这一步在上下文里技术上可接受, 但模糊、低价值、容易误导, 或者不明显推进。

Neutral 的存在很重要。数学推理里不是每句话都能干净地分成对或错。例如一句“我们考虑另一种表示”可能没错，但也不一定有帮助。论文允许 neutral，是为了把人类判断里的模糊性保留下来。主实验里，neutral 在 PRM 打分时按 positive 处理，附录也比较了 neutral 当 positive 或 negative 时的差异，整体差异不大。

数据采集分两个阶段：

- Phase 1 约占 5%，采集多个候选 next step 的标签。这种方式比较笨重，很多候选重复，标注员容易花时间在长而无聊的解法上。
- Phase 2 是主体，先让 generator 生成完整 solutions，再用当前最好的 PRM 排序，优先把“PRM 觉得高分但最终答案错”的解法给标注员。

Phase 2 的主动采样改变了数据分布。它大量收集错误最终答案的解法，但其中仍有很多单步是正确的。这个设计并不是缺陷，而是刻意选择：如果 PRM 被一条错误答案骗了，那么这条解法里至少有一处值得 PRM 学会识别的错误。

5. Active learning 的真实含义

论文里的 convincing wrong-answer solution 是一个非常具体的概念：

- convincing: 当前 PRM 给它高分。
- wrong-answer: 自动检查最终答案发现它错。

这类样本的信息量很高，因为它暴露了当前 PRM 的盲点。随机标注经常会遇到显而易见的错误，标注信号很便宜但学习价值低。主动采样则像专门找模型容易被骗的题。

Small-scale active learning 实验大致是：

1. 用每题一个样本训练一个 PRMselector。
2. 让 PRMselector 给每题 1000 个 samples 打分。
3. 为训练更大的 PRM 选择 N 个 samples。
4. 其中 80% 是 PRMselector 认为最 convincing 的 wrong-answer samples。
5. 另外 20% 是剩余样本里最 convincing 的 samples，避免数据过度偏向错答。
6. 用 PRMlarge 给这些样本提供 step labels。

论文通过比较 learning curve 的斜率，估计这种 active learning 约带来 2.6x 数据效率提升。这个结果的含义不是“主动学习永远 2.6x”，而是在这套 MATH + generator + PRMselector 设置下，优先标高置信错答比均匀采样更划算。

6. PRM 的训练目标

把一条 solution 写成若干步：

```
s = [step_1, step_2, ..., step_T]
```

把完整输入 token 化后，每个 step 有一个结束位置：

```
input_ids shape: [batch, seq_len]
hidden_states shape: [batch, seq_len, hidden_size]
step_end_positions[b] = [pos_1, pos_2, ..., pos_T]
```

PRM 在每个 pos_t 取出最后一层 hidden state：

```
h_t = hidden_states[b, pos_t, :]
logits_t = Linear(h_t)
logits_t shape: [3]
```

三个 logit 对应 positive、negative、neutral。训练目标是标准 cross entropy:

```
loss = average_t CE(logits_t, label_t)
```

这也是论文实现上很聪明的地方: 它没有引入一个奇怪的新训练框架。PRM 本质上还是语言模型上的分类头, 或者等价地说, 让模型在 step 结束处预测一个标签 token。论文说它可以放进标准 LM pipeline 训练。

本仓库的 learning/process-reward/src/prm_minimal.py 就是这个机制的最小实现:

```
class PRMHead(nn.Module):
    def __init__(self, hidden_size: int, num_labels: int = 3):
        super().__init__()
        self.classifier = nn.Linear(hidden_size, num_labels)

    def forward(self, hidden_at_step_end):
        return self.classifier(hidden_at_step_end)
```

score_steps 里的关键句是:

```
h_at_steps = last_h[b, positions]
logits = self.head(h_at_steps)
```

这两行把论文里的 step-end classification 变成了张量操作。学习时一定要亲手追一下 shape, 否则 PRM 很容易被误解成 “给整段文本打一个分” 的普通 verifier。

7. 从 step score 到 solution score

PRM 的输出是一串 step-level probabilities, 但 best-of-N 需要每条 solution 一个总分。论文主策略是:

```
p_t = probability that step_t is positive
score(solution) = product over t of p_t
```

主实验中 neutral 按 positive 处理, 所以更精确地说, 是每一步被认为可接受的概率。乘积的含义是: 一条解法要整体可信, 每一步都不能太差。只要某一步概率很低, 整体分数就会被明显拉低。

这个设计也有副作用: 乘积会偏向步骤更少的解法, 因为多乘几个小于 1 的数会降低总分。附录 F 比较了四种组合:

- neutral 当 positive, 用 product。
- neutral 当 positive, 用 minimum。
- neutral 当 negative, 用 product。
- neutral 当 negative, 用 minimum。

最高的是 neutral 当 positive 加 product, best-of-1860 为 78.2%。其他策略在 77.4% 到 77.8% 左右, 差距不大。论文因此采用 product 作为主策略。

本仓库的 aggregate_step_scores 支持多种聚合方式:

```
def aggregate_step_scores(step_logits, mode="product"):
    probs = F.softmax(step_logits, dim=-1)
    good_prob = probs[:, 0]
    if mode == "mean":
        return good_prob.mean().item()
    if mode == "min":
        return good_prob.min().item()
```

```
if mode == "product":
    return good_prob.prod().item()
```

你可以用一个三步例子感受差异:

```
good_prob = [0.95, 0.92, 0.20]
```

```
mean      = 0.69
min       = 0.20
product   = 0.1748
```

Mean 会把一个严重错误“平均掉”。Minimum 和 product 都会惩罚薄弱步骤。Product 还会表达“每一步都要成立”的联合概率直觉。

8. 为什么只监督到第一处错误

论文刻意只监督到 first incorrect step。也就是说，如果某条解法第三步第一次错了，后面基于错误前提继续推导的步骤不再作为主要监督对象。

这么做有三个理由：

1. 公平比较: Outcome supervision 对错误解法只知道“存在错误”，process supervision 额外知道“第一处错误在哪里”。如果继续标后面的所有错误，process supervision 的信息优势会更大，比较更不保守。
2. 标注成本: 对很多任务来说，判断一条解法是否正确，本质上就是找到第一处错误。
3. 数学结构: MATH 题大多可以认为只要每一步正确，整条解法就正确；一旦某步错，后续推理的局部标签会变得难解释。

这点对新手很重要。PRM 不是在奖励“看起来很长的推理”，而是在学习哪个前缀第一次失去可信度。

9. Best-of-N: reward model 质量如何被放大

Best-of-N 的流程是：

```
for each question:
    sample N solutions from generator
    score each solution with ORM or PRM
    pick solution with highest score
    grade picked solution by final answer
```

如果 generator 本身有一定概率采样出正确解，那么 N 越大，候选池里出现正确解的概率越高。但只有候选池里有正确解还不够，reward model 必须能把它挑出来。

这解释了 Figure 3 为什么关键。论文在 500 个 held-out MATH test problems 上，每题最多采样 1860 条解法。结果是：

- Majority voting: 69.6%。
- ORM: 72.4%。
- PRM: 78.2%。

更重要的是，随着 N 增大，PRM 和 ORM 的差距变大。这说明 PRM 不只是小 N 下运气好，而是在大候选池里更能抵抗“看起来像对但其实错”的解法。ORM 会更容易被最终答案和表面模式误导，PRM 因为检查每一步，能更好地发现隐藏错误。

本仓库 `bon_search.py` 对应这条流程：

```
def best_of_n(candidates, scores):
    idx = int(torch.tensor(scores).argmax().item())
    return candidates[idx], idx, scores[idx]
```

capstone_prm_bon.py 则用 mock 数据对比 greedy、majority、bon、weighted_bon。虽然它不是论文规模实验，但足够让你看懂 best-of-N 的工程接口：generator 产出 candidates，PRM 产出 scores，reranker 选择最终答案。

10. Large-scale 结果: 主证据是什么

Large-scale 实验从 GPT-4 base model 出发。这里的 base model 只做 next-token pretraining，没有 RLHF。作者又做了一个 MathMix 预训练阶段，MathMix 约 1.5B math-relevant tokens，用来增强数学能力。Generator 通过 few-shot 生成 MATH train solutions，过滤最终答案正确的解法，再 finetune 一个 epoch，目的主要是让它输出 newline-delimited step-by-step 格式。

主比较里，ORM 和 PRM 的训练集不完全可比：

- PRM 用 PRM800K 的 step labels 训练，数据来自动态采样，规模较小但标签更细。
- ORM 用 generator 每题 100 个 uniform samples 训练，训练集和 PRM800K 不重叠，规模大约高一个数量级。

论文承认这不是 apples-to-apples comparison，但它代表作者在两种监督方式下各自尽量训练强 reward model 的尝试。这个诚实很重要，因为主结果不能单独证明“在同样数据预算下 PRM 必然赢”。这也是为什么后面要做 small-scale synthetic experiments。

Large-scale 的结论是：在 best-of-N search 中，PRM 明显优于 ORM 和 majority voting，尤其 N 变大时差距扩大。这个结果支持一个机制解释：PRM 更擅长在大量候选中识别局部推理错误。

11. Small-scale synthetic: 更公平的消融

为了更公平地比较 process supervision 和 outcome supervision，论文用 large-scale PRM，记作 PRMlarge，来给小模型生成合成监督信号。这样可以控制数据分布，让同一批 solutions 同时拥有三种标签：

1. PRMlarge 提供的 process supervision。
2. PRMlarge 聚合出来的 outcome supervision。
3. 自动 final-answer checking 提供的 outcome supervision。

Appendix H 给了关键阈值：如果 PRMlarge 给某一步 negative label 的概率大于 20%，就把这一步视为 incorrect。Process supervision 返回直到第一处 incorrect step 的 step-level labels。Outcome supervision 则把整条 solution 标为正确，当且仅当 PRMlarge 认为每一步都正确。

Figure 4a 显示，在不同训练数据规模下，process supervision 都显著优于两种 outcome supervision。Figure 4b 显示，当固定训练规模、改变 test-time N 时，process-supervised PRM 仍然更强。

这组实验是论文证据链中最重要的消融。它排除了两个混杂因素：

- PRM800K 的主动采样数据分布和 ORM 的均匀采样数据不同。
- 自动 final-answer checking 会把“歪打正着”的错误推理标成正例。

消融后的结论仍然支持 process supervision：不是因为 PRM 拿到了更幸运的数据，而是 step-level labels 本身提供了更有效的学习信号。

12. OOD 和限制: 论文证明了什么, 没有证明什么

论文还在近期 STEM 测试题上做了 out-of-distribution evaluation, 包括 AP Physics、AP Calculus、AP Chemistry、AMC10/12 等。Aggregate 结果中, PRM 为 72.9%, ORM 为 63.8%, majority voting 为 61.3%。这说明 PRM 的优势不是只在 MATH held-out subset 上出现。

但论文没有证明以下事情:

- PRM 一定能泛化到所有推理领域, 例如法律、医学、开放式科研。
- Step-level human labels 在任何任务上都划算。
- PRM 用于 RL 训练 generator 时一定稳定。
- PRM 能完全解决 reward hacking。
- MATH test contamination 对绝对数值没有影响。

作者也讨论了 test set contamination。MATH test problems 可能出现在互联网文本中, MathMix 尽管做了字符串过滤, 也不能保证移除所有重述版本。论文的防守理由是: 相对比较可能仍然成立; 人工检查没看到明显 memorization; 近期 STEM OOD 结果也支持 PRM 优势。

所以正确读法是: 这篇论文强力证明了“在数学 best-of-N reranking 中, step-level process supervision 训练出的 reward model 更可靠”。它没有完成“所有领域的 process supervision 都比 outcome supervision 好”的终局证明。

13. Alignment 论证: negative alignment tax

论文讨论了一个很有影响力的观点: process supervision 可能具有 negative alignment tax。

Alignment tax 指更安全的方法牺牲了性能, 导致工程上更难被采用。这里作者观察到, process supervision 不但更符合人类可检查的推理过程, 而且在 MATH 上性能更强。因此它不是“为了安全牺牲能力”, 而是“更安全的监督形式同时提升能力”。

这句话要谨慎读。论文的 alignment 论证主要是基于数学推理的 evidence:

- PRM 奖励人类认可的中间步骤。
- PRM 让错误位置更可解释。
- ORM 可能鼓励只优化最终结果, 甚至学会利用 outcome proxy。

但它还不是通用安全证明。复杂开放任务里的“好过程”未必容易定义, 标注员也可能被流畅但错误的过程骗过。现代 reasoning system 仍然需要 verifier、tool use、formal checks、RLVR、过程可视化和反作弊评估共同配合。

14. 方法总图

PRM800K data collection

```
MATH problem
|
v
large generator samples step-by-step solution
|
v
human labeler marks each step
|
+-- positive: correct and useful
+-- neutral : acceptable but ambiguous or low progress
```



```

    +-- negative: wrong or unreasonable
    |
    v
train PRM to classify labels at step-end positions

```

Evaluation

```

MATH test problem
|
v
fixed generator samples N candidate solutions
|
v
PRM scores each step in each solution
|
v
aggregate step probabilities into solution score
|
v
select highest-scoring solution
|
v
automatic final-answer grader computes accuracy

```

15. 张量级别图示

One batch item:

tokens:

```
[Q tokens ... step1 tokens \n step2 tokens \n step3 tokens \n]
```

positions:

```

          p1          p2          p3
          |           |           |

```

hidden:

```

H shape = [seq_len, hidden_size]
H[p1] -> Linear -> logits_1 shape [3]
H[p2] -> Linear -> logits_2 shape [3]
H[p3] -> Linear -> logits_3 shape [3]

```

labels:

```

label_1 = positive
label_2 = positive
label_3 = negative

```

loss:

```

CE(logits_1, label_1)
+ CE(logits_2, label_2)
+ CE(logits_3, label_3)

```

then average over labeled steps

```
inference:
softmax(logits_t)[positive] = p_t
solution_score = p_1 * p_2 * p_3
```

新手最容易错的是把 seq_len 维度上的每个 token 都当作一个 step。论文关心的是 step end positions。一个 step 可能有很多 token，但 PRM 标签落在这个 step 的结束位置。

16. 代码样例: 最小 PRM 聚合

下面的代码不是复现论文规模，而是复现论文的核心数据流：

```
import torch
import torch.nn.functional as F

# 3 steps, 3 labels: positive, negative, neutral
logits = torch.tensor([
    [3.0, 0.1, 0.0],
    [2.5, 0.2, 0.1],
    [0.2, 2.8, 0.0],
])

probs = F.softmax(logits, dim=-1)
p_positive = probs[:, 0]

mean_score = p_positive.mean()
min_score = p_positive.min()
product_score = p_positive.prod()

print(p_positive)
print(mean_score, min_score, product_score)
```

你应该观察到第三步的 positive probability 很低，product score 会被强烈压低。这就是 PRM 的直觉：一条多步推理链只要有一个关键步骤不可信，整体就不该高分。

17. 和 Math-Shepherd、PRIME、RLVR 的关系

本专题的后续代码故意把 PRM 放到更大的 reasoning reward map 里。

math_shepherd_data_gen.py 对应后来的自动过程标注路线。它不依赖人类逐步标注，而是从某个 step prefix 出发做多次 rollout。如果后续 rollout 大多到达正确答案，就把这个 step 标成 good；如果大多失败，就标成 bad；中间就是 neutral。这是在降低 PRM800K 式人工标签成本。

prime_minimal.py 对应 implicit PRM 思路。它用 actor 和 reference model 的 log probability 差作为 token-level reward，再聚合到 step-level。这条路线想绕开显式 PRM 数据。

rlvr_demo.py 对应 verifiable reward。数学、代码、形式化证明等任务可以用规则或程序检查最终结果。RLVR 的优势是客观、便宜、可自动化；缺点是它仍然偏 outcome，可能不知道中间哪一步错。PRM 和 RLVR 的关系不是互斥，而是互补：RLVR 给可靠终局信号，PRM 给过程信用分配。

mcts_llm.py 对应 search-time reasoning。PRM 不只可以做 final reranking，也可以在树搜索中作为 value signal，帮助选择下一步展开哪条推理路径。

18. 和本仓库的连接

建议按下面顺序学习本专题:

1. 读 `learning/process-reward/lectures/01-orm-vs-prm.md`, 先建立 ORM 与 PRM 的差异。
2. 读本 `guide`, 对照原论文 PDF 的 Figure 3、Figure 4、Appendix F。
3. 打开 `learning/process-reward/src/prm_minimal.py`, 追踪 step-end hidden 到 classifier logits。
4. 打开 `learning/process-reward/src/bon_search.py`, 理解 best-of-N、majority vote、weighted BoN。
5. 运行 `learning/process-reward/src/capstone_prm_bon.py`, 观察 mock PRM rerank 为什么能高于 greedy。
6. 再看 `math_shepherd_data_gen.py`、`prime_minimal.py`、`rlvr_demo.py`, 把后续路线接上。

一个 30-60 分钟本地实验:

目标: 比较 mean、min、product 三种 PRM 聚合方式。

步骤:

1. 在 `prm_minimal.py` 里构造 3 条 toy solutions。
2. 第一条每一步都高分。
3. 第二条前两步高分, 最后一步低分。
4. 第三条很多步中等分。
5. 分别用 mean、min、product 聚合。
6. 再用 `bon_search.py` 选择最高分 solution。

预期:

mean 更容易放过“一步严重错误”的解法。

min 对任意低分步骤很敏感。

product 同时惩罚低分步骤和过长步骤。

这个实验能让你把论文 Appendix F 的 scoring strategy 变成手感, 而不是只记数字。

19. AI agent 应该怎样辅助你学这篇

这篇论文非常适合用 agent 加速, 但前提是你不能让 agent 只总结。你应该让 agent 做三类事:

第一, 让 agent 扮演 examiner。它一次只问一个问题, 例如“为什么 PRM 的 best-of-N 差距会随 N 增大而扩大”。你自己先回答, 再让 agent 挑错。

第二, 让 agent 把论文和代码强绑定。任何概念都要落到文件和函数, 例如:

请把论文里的 step-end classification 对应到

`learning/process-reward/src/prm_minimal.py` 的具体函数。

不要泛泛解释, 必须说明 `input_ids`、`hidden_states`、`step_end_positions`、`logits_list` 的 shape。

第三, 让 agent 帮你设计反例。比如让它构造一条最终答案正确但中间推理错误的 solution, 再问 ORM 和 PRM 会分别学到什么。这类反例比摘要更能进脑子。

一个推荐提示词:

我正在学 Lightman et al. 2023 的 Let's Verify Step by Step。

请你按 examiner 模式考我。一次只问一个问题。

每个问题必须要求我同时解释论文机制和本仓库代码。

如果我回答太泛，请追问 shape、label、loss 或实验数字。
最后让我闭卷画出 ORM vs PRM 的 best-of-N 流程图。

20. 常见误读

误读一：PRM 是让模型生成更详细的 CoT。

更准确：PRM 是训练一个 reward model 检查每一步。它不直接保证 generator 写得更长或更好。

误读二：Outcome supervision 没有用。

更准确：ORM 仍然强于 majority voting，但在复杂多步推理里 credit assignment 更难。论文结论是 PRM 更可靠，不是 ORM 毫无价值。

误读三：PRM800K 的标签全是正确解法。

更准确：Phase 2 主动采样大量最终答案错误但 PRM 觉得 convincing 的解法。数据里很多 solution 是 wrong-answer，但里面仍有大量正确步骤。

误读四：PRM 的 solution score 只能用乘积。

更准确：乘积是论文主策略，附录比较了 minimum 和 neutral 处理方式。差异不大，但乘积最强。

误读五：78.2% 是 generator 单次解题准确率。

更准确：78.2% 是 PRM 在 best-of-1860 reranking 后，在代表性 MATH test subset 上挑出的最终答案正确率。

21. 现代意义

今天看这篇论文，它是 reasoning reward model 的奠基论文之一。后来的很多方向都能从这里接出去：

- PRM 数据如何便宜采集：Math-Shepherd、自动 rollout 标注、AI feedback。
- PRM 如何和搜索结合：best-of-N、beam search、tree search、MCTS。
- PRM 如何和 RL 结合：process reward shaping、step-level advantage、verifier-guided policy optimization。
- PRM 如何和 RLVR 结合：outcome verifier 提供硬终局，PRM 提供软过程信号。
- PRM 如何避免被 hack：adversarial candidate mining、active learning、calibration、OOD verifier eval。

这篇论文对现在的意义不是“所有系统都该人工标 PRM800K”，而是提供了一个标准问题表述：如果任务需要多步推理，只奖励最终答案会让 credit assignment 过难；更细粒度的过程监督可以同时提升能力、可解释性和某些安全属性。

22. 闭卷掌握检查

读完后你应该能闭卷回答：

1. 为什么论文说 generator 没有被 RL 训练，这个范围限定为什么重要。
2. ORM 和 PRM 的训练标签分别是什么，测试时分别用哪个位置的分数。
3. PRM800K 的 positive、negative、neutral 分别代表什么。
4. 为什么主动采样 convincing wrong-answer solutions 信息量高。
5. 为什么 process supervision 只标到第一处错误。
6. $\text{score}(\text{solution}) = \text{product}_t p_t$ 的概率直觉是什么，它有什么偏差。
7. Figure 3 的 78.2%、72.4%、69.6% 分别是谁，说明了什么。
8. Figure 4 的 synthetic supervision 排除了哪些混杂因素。

9. Appendix H 里 PRMlarge 怎样把 step probabilities 转成合成标签。
10. 为什么这篇论文能支持 negative alignment tax 的说法，但不能构成通用安全证明。
11. 在 `prm_minimal.py` 中，step-end hidden state 是在哪一行被取出的。
12. 如果 mean 聚合选中了一条有严重错误步骤的 solution，你会如何解释这个失败。

真正掌握的标志是：你能画出从 human step labels 到 PRM logits，再到 best-of-N selection 的完整流程，并能用本仓库 50 行以内的 toy code 复现 “一个低分步骤拉低整条解法” 的现象。